

Assignment Title:
C Libraries, Linking, and ELF Executable Structure

Coursework Type:
Individual

Module Name:
ST5003CMD – Operating Systems, Security, And Networks

Submitted By:
Name: Pranisha Bhatta
College ID: 250150

Module Leader:
Name: Rupak Rajbanshi

Table of content

Lab 2: C Libraries, Linking, and ELF Executable Structure **3**
Objective 3
Task 1: Create and Run the Program **3**
Step 1: Create and edit the file **3**
Step 2: Compile and run **4**
Task 2: Header Files **4**
Task 3: Library Files **4**
Task 4: Static vs Dynamic Linking **5**
Task 5: Inspecting the ELF File with readelf **5**
Step 1: ELF Header **6**
Step 2: Program Headers **7**
Step 3: Dynamic Section **8**
Task 6: Tracing Dependencies with ldd **8**
Conclusion **10**

Lab 2: C Libraries, Linking, and ELF Executable Structure

Objective

To understand how a C program links to the C Standard Library, where library files are stored, the difference between static and dynamic linking, and how to inspect executables using `readelf` and `ldd`.

Task 1: Create and Run the Program

Step 1: Create and edit the file

```
pranisha@ubuntu:~/Desktop$ touch procinfo.c
pranisha@ubuntu:~/Desktop$ nano procinfo.c
```

```
  GNU nano 8.7.1                                        procinfo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    // Format and display information
    printf("=== Process Information ===\n");
    printf("My Process ID (PID):  %d\n", my_pid);
    printf("Parent Process ID:    %d\n", my_ppid);
    printf("Current Time:         %s", asctime(time_info));
    printf("Executable Path:      /proc/self/exe\n");

    return 0;
}
```

`touch` created an empty C file and `nano` opened it for editing. The program uses `getpid()`, `getppid()`, `time()`, `localtime()`, `printf()` and `asctime()` to show process information.

Step 2: Compile and run

```
pranisha@ubuntu:~/Desktop$ gcc procinfo.c -o procinfo
pranisha@ubuntu:~/Desktop$ ./procinfo
=== Process Information ===
My Process ID (PID):  3848
Parent Process ID:    3730
Current Time:         Sun Oct  4 20:33:49 2026
Executable Path:      /proc/self/exe
```

`gcc` compiled the source into an executable named `procinfo`. Running it printed the process ID, parent process ID, current time and executable path. This shows the program successfully used library functions.

Task 2: Header Files

```
pranisha@ubuntu:~/Desktop$ ls /usr/include/stdio.h /usr/include/unistd.h
/usr/include/stdio.h  /usr/include/unistd.h
```

The header files exist in `/usr/include/`, which is where the compiler looks when it sees `#include`.

```
pranisha@ubuntu:~/Desktop$ grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN.  */
extern int fprintf (FILE *__restrict __stream,
pranisha@ubuntu:~/Desktop$ grep "getpid" /usr/include/unistd.h | head -2
extern __pid_t getpid (void) __THROW;
```

The output shows only function declarations (prototypes) for `printf` and `getpid`. Headers tell the compiler what a function looks like but contain no executable code.

Task 3: Library Files

```
pranisha@ubuntu:~/Desktop$ ls /lib/*-linux-gnu/libc.so*
/lib/aarch64-linux-gnu/libc.so  /lib/aarch64-linux-gnu/libc.so.6
pranisha@ubuntu:~/Desktop$ ls /usr/lib/*-linux-gnu/libc.a
/usr/lib/aarch64-linux-gnu/libc.a
```

The actual compiled code of the C library is stored here. `libc.so.6` is the shared (dynamic) library and `libc.a` is the static library. Their folder name depends on the system architecture (e.g. `x86_64-linux-gnu` or `aarch64-linux-gnu`).

Task 4: Static vs Dynamic Linking

```
pranisha@ubuntu:~/Desktop$ gcc procinfo.c -o procinfo_dynamic
pranisha@ubuntu:~/Desktop$ gcc -static procinfo.c -o procinfo_static
pranisha@ubuntu:~/Desktop$ ls -lh procinfo_dynamic procinfo_static
-rwxrwxr-x 1 pranisha pranisha  70K Oct  4 21:38 procinfo_dynamic
-rwxrwxr-x 1 pranisha pranisha 769K Oct  4 21:38 procinfo_static
```

The same program was compiled in two ways. The static binary is much larger because the library code is copied into it, while the dynamic binary is small because it only references `libc.so.6` and loads it at runtime.

Task 5: Inspecting the ELF File with readelf

Step 1: ELF Header

```
pranisha@ubuntu:~/Desktop$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           AArch64
  Version:                           0x1
  Entry point address:               0x900
  Start of program headers:          64 (bytes into file)
  Start of section headers:          68944 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         10
  Size of section headers:           64 (bytes)
  Number of section headers:         29
  Section header string table index: 28
```

The Magic bytes `7f 45 4c 46` confirm this is a valid ELF file. Class shows ELF64 (64-bit), and Machine shows the CPU type the binary was built for. Type is DYN, meaning position-independent.

Step 2: Program Headers

```
pranisha@ubuntu:~/Desktop$ readelf -l procinfo_dynamic

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x900
There are 10 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x0000000000000230 0x0000000000000230  R      0x8
  INTERP         0x0000000000000294 0x0000000000000294 0x0000000000000294
                 0x000000000000001b 0x000000000000001b  R      0x1
      [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
  LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000cd4 0x0000000000000cd4  R E    0x10000
  LOAD           0x000000000000fd40 0x000000000001fd40 0x000000000001fd40
                 0x00000000000002d0 0x00000000000002d8  RW     0x10000
  DYNAMIC        0x000000000000fd50 0x000000000001fd50 0x000000000001fd50
                 0x0000000000000200 0x0000000000000200  RW     0x8
  NOTE           0x0000000000000270 0x0000000000000270 0x0000000000000270
                 0x0000000000000024 0x0000000000000024  R      0x4
  NOTE           0x0000000000000cb4 0x0000000000000cb4 0x0000000000000cb4
                 0x0000000000000020 0x0000000000000020  R      0x4
  GNU_EH_FRAME   0x0000000000000bc0 0x0000000000000bc0 0x0000000000000bc0
                 0x000000000000003c 0x000000000000003c  R      0x4
  GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
```

`INTERP` gives the path of the dynamic linker that loads before `main()`. The `LOAD` segments show memory regions with their permissions: R (read-only), R E (executable code) and RW (data). `GNU_STACK` shows RW, so the stack is not executable, which prevents stack-based buffer overflow attacks.

Step 3: Dynamic Section

```
pranisha@ubuntu:~/Desktop$ readelf -d procinfo_dynamic

Dynamic section at offset 0xfd50 contains 28 entries:
  Tag        Type                         Name/Value
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
 0x0000000000000001 (NEEDED)             Shared library: [ld-linux-aarch64.so.1]
 0x000000000000000c (INIT)               0x7e0
 0x000000000000000d (FINI)               0xaf8
 0x0000000000000019 (INIT_ARRAY)         0x1fd40
 0x000000000000001b (INIT_ARRAYSZ)       8 (bytes)
 0x000000000000001a (FINI_ARRAY)         0x1fd48
 0x000000000000001c (FINI_ARRAYSZ)       8 (bytes)
 0x000000006ffffef5 (GNU_HASH)           0x2b0
 0x0000000000000005 (STRTAB)             0x480
 0x0000000000000006 (SYMTAB)             0x2d0
 0x000000000000000a (STRSZ)              243 (bytes)
 0x000000000000000b (SYMENT)             24 (bytes)
 0x0000000000000015 (DEBUG)              0x0
 0x0000000000000003 (PLTGOT)             0x1ff50
 0x0000000000000002 (PLTRELSZ)           288 (bytes)
 0x0000000000000014 (PLTREL)             RELA
 0x0000000000000017 (JMPREL)             0x6c0
 0x0000000000000007 (RELA)               0x5e8
 0x0000000000000008 (RELASZ)             216 (bytes)
 0x0000000000000009 (RELAENT)            24 (bytes)
 0x000000000000001e (FLAGS)              BIND_NOW
 0x000000006ffffffb (FLAGS_1)            Flags: NOW PIE
```

The NEEDED entry lists `libc.so.6`, the shared library the program requires. The dynamic linker loads it into memory before `main()` starts, and the program will not run if it is missing.

Task 6: Tracing Dependencies with ldd

```
pranisha@ubuntu:~/Desktop$ ldd procinfo_dynamic
        linux-vdso.so.1 (0x0000e3fee23ce000)
        libc.so.6 => /usr/lib/aarch64-linux-gnu/libc.so.6 (0x0000e3fee2180000)
        /lib/ld-linux-aarch64.so.1 (0x0000e3fee2380000)
```

`ldd` shows the full path of each required library. It lists `libc.so.6`, the dynamic linker, and `linux-vdso.so.1`, a virtual library the kernel adds to every process for fast system calls.

```
pranisha@ubuntu:~/Desktop$ ldd procinfo_static
        not a dynamic executable
```

The output "not a dynamic executable" confirms the static binary has no runtime library dependencies.

Knowledge Check Questions

1. What is the fundamental difference between a C header file and a shared library?

A header file (`.h`) contains declarations only, such as prototypes, macros and types. It tells the compiler what functions exist. A shared library (`.so`) contains the actual compiled machine code. Headers are used at compile time and shared libraries at runtime.

2. What is static and dynamic linking?

Static linking copies the needed library code (`.a`) into the executable at compile time, giving a large but self-contained file. Dynamic linking keeps only references to shared libraries (`.so`), which the dynamic linker loads at runtime, giving a small file where many programs share one copy in memory.

3. What is ELF Header? Write down the commands to check headers.

The ELF Header is the first part of an ELF file and holds global metadata: whether the file is valid, 32 or 64-bit, endianness, file type, target CPU and entry point address.

Purpose	Command
ELF Header	readelf -h <file>
Program Headers	readelf -l <file>
Dynamic Section	readelf -d <file>

4. What does bookmark mean? Provide example.

A bookmark is a saved shortcut to a location so you can return to it quickly. Example: a browser bookmark opens a saved web page in one click. In Linux, an alias can act as a bookmark for a folder, e.g. `alias labs='cd ~/os/labs'`.

Conclusion:

The C Standard Library has two parts: headers (declarations) in `/usr/include/` and compiled code in `/lib/` and `/usr/lib/`. Static linking produces large self-contained binaries, while dynamic linking produces small binaries that share libraries at runtime. `readelf` and `ldd` let us inspect an executable's header, memory layout and dependencies.